agronomy

MDPI

Article

YOLO-YSTs: An Improved YOLOv10n-Based Method for Real-Time Field Pest Detection

Yiqi Huang [1,†], Zhenhao Liu [1,2,†], Hehua Zhao [3], Chao Tang [4], Bo Liu [2], Zaiyuan Li [2], Fanghao Wan [2], Wanqiang Qian [2] and Xi Qiao [2,*]

1 College of Mechanical Engineering, Guangxi University, Nanning 530004, China; hyq@gxu.edu.cn (Y.H.); 2211301042@st.gxu.edu.cn (Z.L.)
2 Shenzhen Branch, Guangdong Laboratory for Lingnan Modern Agriculture, Genome Analysis Laboratory of the Ministry of Agriculture and Rural Affairs, Agricultural Genomics Institute at Shenzhen, Chinese Academy of Agricultural Sciences, Shenzhen 518120, China
3 College of Mechanical and Electrical Engineering, Qingdao Engineering Vocational College, Qingdao 266112, China
4 Environment and Plant Protection Research Institute, Chinese Academy of Tropical Agricultural Sciences, Haikou 571701, China
* Correspondence: qiaoxi@caas.cn
† These authors contributed equally to this work.

Abstract: The use of yellow sticky traps is a green pest control method that utilizes the pests' attraction to the color yellow. The use of yellow sticky traps not only controls pest populations but also enables monitoring, offering a more economical and environmentally friendly alternative to pesticides. However, the small size and dense distribution of pests on yellow sticky traps lead to lower detection accuracy when using lightweight models. On the other hand, large models suffer from longer training times and deployment difficulties, posing challenges for pest detection in the field using edge computing platforms. To address these issues, this paper proposes a lightweight detection method, YOLO-YSTs, based on an improved YOLOv10n model. The method aims to balance pest detection accuracy and model size and has been validated on edge computing platforms. This model incorporates SPD-Conv convolutional modules, the iRMB inverted residual block attention mechanism, and the Inner-SIoU loss function to improve the YOLOv10n network architecture, ultimately addressing the issues of missed and false detections for small and overlapping targets while balancing model speed and accuracy. Experimental results show that the YOLO-YSTs model achieved precision, recall, mAP_{50}, and $mAP_{50–95}$ values of 83.2%, 83.2%, 86.8%, and 41.3%, respectively, on the yellow sticky trap dataset. The detection speed reached 139 FPS, with GFLOPs at only 8.8. Compared with the YOLOv10n model, the mAP_{50} improved by 1.7%. Compared with other mainstream object detection models, YOLO-YSTs also achieved the best overall performance. Through improvements to the YOLOv10n model, the accuracy of pest detection on yellow sticky traps was effectively enhanced, and the model demonstrated good detection performance when deployed on edge mobile platforms. In conclusion, the proposed YOLO-YSTs model offers more balanced performance in the detection of pest images on yellow sticky traps. It performs well when deployed on edge mobile platforms, making it of significant importance for field pest monitoring and integrated pest management.

Keywords: yellow sticky traps; pest detection; Yolov10n; lightweight; edge computing platforms

check for updates

Academic Editor: Jayme Garcia Arnal Barbedo

Received: 24 January 2025
Revised: 17 February 2025
Accepted: 24 February 2025
Published: 26 February 2025

Citation: Huang, Y.; Liu, Z.; Zhao, H.; Tang, C.; Liu, B.; Li, Z.; Wan, F.; Qian, W.; Qiao, X. YOLO-YSTs: An Improved YOLOv10n-Based Method for Real-Time Field Pest Detection. *Agronomy* **2025**, *15*, 575. https://doi.org/10.3390/agronomy15030575

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Yellow sticky traps are a common physical pest control tool, primarily used in agriculture and horticulture [1]. These traps exploit insects' attraction to the yellow wavelength, attracting small flying pests such as aphids and whiteflies. The sticky surface traps and kills them, thus reducing pest population density [2]. By regularly monitoring the types and quantities of pests adhered to the traps, staff can assess pest infestation trends, providing a basis for targeted pesticide application or other control measures [3–5]. As a green pest control method, yellow sticky traps reduce the use of chemical pesticides. They are simple to operate, cost-effective, and serve dual functions in both pest control and monitoring, making them an important tool in integrated pest management (IPM) [6,7]. Traditionally, pest identification on sticky traps is performed manually by human eyes, a process that requires substantial labor and time. Additionally, it demands specialized knowledge from workers, resulting in high labor costs and a risk of misidentification [8]. In recent years, numerous innovative technologies have emerged in the field of pest detection, with deep learning-based pest detection methods developing rapidly. Böckmann et al. [9] used the bag of visual words (BoVW) algorithm to differentiate between the natural enemies *Trialeurodes vaporariorum* and *Bemisia tabaci* species. Xia et al. [10] used the VGG19 network as a feature extractor in combination with a region proposal network (RPN) to detect insects on plant leaves in farmland. Gutierrez et al. [11] evaluated the efficiency of computer vision and deep learning algorithms in pest detection in tomato farms. Ramalingam et al. [12] applied the faster R-CNN ResNet object detection framework to detect agricultural pests. In the task of object detection on yellow sticky traps, Deserno et al. [13] used faster R-CNN and EfficientNet to detect pest images captured on the traps. Xia et al. [14] used sticky traps to capture whiteflies, aphids, and thrips in greenhouses, followed by identification. These studies have all contributed to advancements in the field of pest detection. However, these studies have not yet addressed the detection of small-sized and densely distributed pests, nor have they been deployed on edge computing platforms for testing. Therefore, there is a need to develop new models to solve these issues.

Deep learning techniques, particularly the YOLO algorithm, have demonstrated exceptional performance in object detection. YOLO performs both object localization and classification through a single network, enabling end-to-end training and achieving both accuracy and real-time performance. When applied to the field of pest detection, YOLO also outperforms other methods, demonstrating its versatility. In the field of pest detection, the YOLO model has improved detection accuracy, achieved lightweight performance, and is a reliable tool for pest detection [15–29]. For instance, Sun et al. [30] proposed an improved YOLO-FMDI model for tomato pest and disease detection, reducing model parameters while maintaining high accuracy. YOLOv10, released in May 2024 as a version of the YOLO series, has demonstrated outstanding performance and efficiency [31]. Prior to YOLOv10, the YOLO series' reliance on non-maximum suppression (NMS) in post-processing hindered end-to-end deployment and affected inference speed. YOLOv10 introduced an NMS-free strategy with a dual allocation approach and consistent matching metrics to address the issue of redundant predictions in post-processing. This eliminates NMS dependency and performs well on large-object tasks [32]. However, YOLOv10 may encounter issues such as missed detections, false positives, or overlapping detections when detecting small and dense targets. For example, Guan et al. [33] encountered issues such as missed detections when using YOLOv10 for wheat ear detection, particularly in dense and complex backgrounds. This indicates that YOLOv10 requires improvements to address such issues effectively.

To address the issues of missed detections, false positives, and overlapping detections of small and dense targets, researchers have introduced modules such as SPD-Conv, iRMB

attention mechanisms, and the Inner-SIoU loss function to enhance model performance. SPD-Conv is an innovative spatial encoding method that enhances deep learning model performance by processing image data more efficiently [34]. This technique converts spatial information in images into depth information and improves feature extraction accuracy by reducing information loss. It optimizes the model's performance when handling small-target images. For example, Sun et al. [35] proposed a drone inspection small-target detection algorithm based on SPD-Conv to address the challenges of detecting small and densely distributed targets during drone inspections, achieving improved accuracy for small-target detection. Peng et al. [36] replaced the convolutional layers in the YOLOv8 backbone network with SPD-Conv, improving the recognition rate of elongated and small cracks in road crack detection. The core concept of iRMB is to combine lightweight convolutional neural network (CNN) architectures with attention-based model structures, aiming to create efficient mobile networks [37]. For instance, Lu et al. [38] integrated the iRMB attention mechanism into the YOLOv5s model, effectively improving small-target detection accuracy in paint defect detection. The core concept of Inner-IoU is to improve bounding box regression accuracy in object detection, particularly in cases involving high target overlap [39]. The SIoU loss function introduces a more complex bounding box regression method by incorporating angle consideration and scale sensitivity, addressing the limitations of previous loss functions [40]. By combining the concepts of Inner-IoU and SIoU to form Inner-SIoU, the model maximizes its effectiveness in small-target detection. For example, Chen et al. [41] improved the YOLOv8n model with Inner-SIoU, enhancing accuracy in detecting the appearance defects of power adapters.

In addition, current pest monitoring devices on the market are large, expensive (often exceeding 100,000 RMB), and have high deployment and maintenance costs in complex field environments. Therefore, developing compact, low-cost, and easy-to-install small-scale pest monitoring platforms holds great potential for application [42]. Low-cost monitoring platforms inherently have limited computational resources, which places higher demands on the lightweight nature and accuracy of object detection models.

Based on the above research background, this paper aims to enhance the detection capability for small and dense targets by improving the YOLOv10n model, thereby enabling pest detection on yellow sticky traps. At the same time, an intelligent pest monitoring device is developed, using the Raspberry Pi 4B as the processor unit and a USB camera as the image acquisition unit to detect pests on yellow sticky traps. The Raspberry Pi 4B features excellent hardware configuration and high computational performance, making it versatile in many use cases. Compared with other edge computing devices, such as the NVIDIA Jetson Nano, the Raspberry Pi 4B offers comparable overall performance. However, the Raspberry Pi 4B is significantly more affordable than the NVIDIA Jetson Nano and has considerable resource advantages, including comprehensive development documentation and reliable after-sales support. Edge computing devices with configurations equal to or higher than the Raspberry Pi 4B can also achieve the functionality described in this paper. However, considering cost and development investment, we chose the Raspberry Pi 4B as the processing unit. The specific improvements to the YOLO-YSTs pest detection model for yellow sticky traps are obtained as follows: 1. Through the use of the SPD-Conv module to replace the stride convolutions and pooling operations in the YOLOv10n model, thereby avoiding information loss from downsampling while reducing the number of parameters. 2. By adding the iRMB inverted residual block attention mechanism to the YOLOv10n model to balance the model's focus on local detail features and global semantic features. 3. By using the Inner-SIoU loss function in the YOLOv10n model to address the issues of missed and false detections for small and overlapping targets. The improved model

is intended to be deployed on the Raspberry Pi 4B, enabling field-based intelligent pest monitoring and providing technical support for integrated agricultural pest management.

2. Materials and Methods

2.1. Pest Detection Device

The pest detection device in this paper employs the Raspberry Pi 4B as the processing unit and a USB camera as the image acquisition module. The Raspberry Pi 4B is an embedded computer based on the Linux operating system, characterized by its compact size and strong computational power. The USB camera is easy to install and provides clear imaging. The model proposed in this paper is deployed on the Raspberry Pi 4B. The Raspberry Pi 4B, USB camera and pest detection device are shown in Figure 1, and the specific parameters of the Raspberry Pi 4B are provided in Table 1. The Raspberry Pi 4B is equipped with a rich set of GPIO interfaces and supports a range of mainstream frameworks and algorithms. The software development environment for this device comprises the Raspbian operating system, Python 3.9, and OpenCV 4.1, and utilizes NCNN for inference.

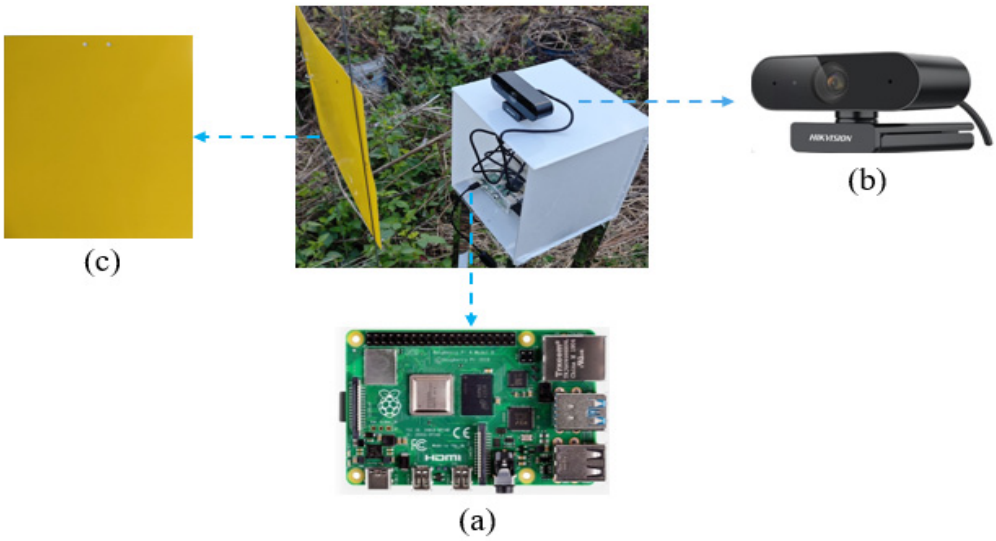

Figure 1. The pest detection device: (**a**) Raspberry Pi 4 Model B; (**b**) USB camera; (**c**) Yellow sticky traps.

Table 1. Raspberry Pi 4 Model B equipment parameters.

Parameter	Technical Specifications
Processor	Broadcom BCM2711, quad-core Cortex-A72 (ARM v8) 64-bit SoC @ 1.5GHz
Memory	8GB LPDDR4 with on-die ECC
Connectivity:	2.4 GHz and 5.0 GHz IEEE 802.11b/g/n/ac wireless LAN, Bluetooth 5.0, BLE
GPIO	Standard 40-pin GPIO header
SD card support	Micro SD card slot for loading operating system and data storage
Input power	5V DC via USB-C connector (minimum 3A)

2.2. Dataset

The dataset in this study is derived from pest images collected from yellow sticky traps placed in the field. To accelerate the collection of a sufficient number of images, the collection environment was simulated to align with the pest detection device, ensuring

consistent lighting, shooting distance, and resolution. Yellow sticky traps were placed at different locations in the field, and pest images were captured every 1–2 days. After each session, the used traps were removed and replaced with new ones for continuous pest collection, followed by subsequent image capture every 1–2 days. The equipment used was a Xiaomi 14 smartphone, and the captured images were saved in JPG format with a resolution of 2048 × 1080. In the experiment, image data were collected using a smartphone for model training, and USB cameras were used on edge computing devices for validation. This approach allows one to validate the improved model's pest detection capability under various device conditions. The use of smartphones for data collection was aimed at quickly obtaining the basic data required for model training. After preprocessing, such as pixel compression, the quality of the smartphone-collected data was consistent with that of images collected by commercial cameras, effectively eliminating issues of inconsistent spatial resolution caused by different devices. The GPS coordinates of the collection site were 22.6° N, 114.5° E, located in an agricultural field in the wilderness, with the collection period spanning from March 2024 to August 2024. The collection method involved using a smartphone to take pictures of the yellow sticky traps, which were removed after capturing pests, at a height of 20 to 30 cm and at a vertical angle. No pesticides were used during the collection period to intervene with the field insects. The pests on the yellow sticky traps were naturally attracted due to the trap's insect-attracting properties. The field collection results are shown in Figure 2.

(**a**) (**b**)

Figure 2. Field pest acquisition images: (**a**) Acquisition mode; (**b**) pest image.

Additionally, due to the monoculture crops in the experimental field, the pest images collected from the field primarily consisted of *Whiteflies* (WF), *Flea beetles* (SFB), and *Syrphidae* (SP). To enhance the model's generalization ability for multi-class pest detection and improve its robustness, this paper also includes the public dataset yellow sticky traps as a supplement. This dataset, consisting of 284 images, was created by the 4TU federation, which includes four renowned Dutch technical universities (Delft University of Technology, Eindhoven University of Technology, University of Twente, and Wageningen University), and is intended for pest detection research. The dataset includes three pest categories: *Whiteflies* (WF), *Macrolophus* (MR), and *Nesidiocoris* (NC), with pest images shown in Figure 3. Figure 3 shows that the pests on the sticky traps are relatively small. The size of *Macrolophus* is typically between 50 and 80 pixels, *Flea beetles* range from 60 to 110 pixels, *Nesidiocoris* sizes range from 80 to 100 pixels, and *Syrphidae* are approximately 200 pixels in size. The smallest pests, *Whiteflies*, range from 10 to 40 pixels in size, and *Flea beetles* and *Whiteflies* exhibit dense distributions and overlap in certain areas.

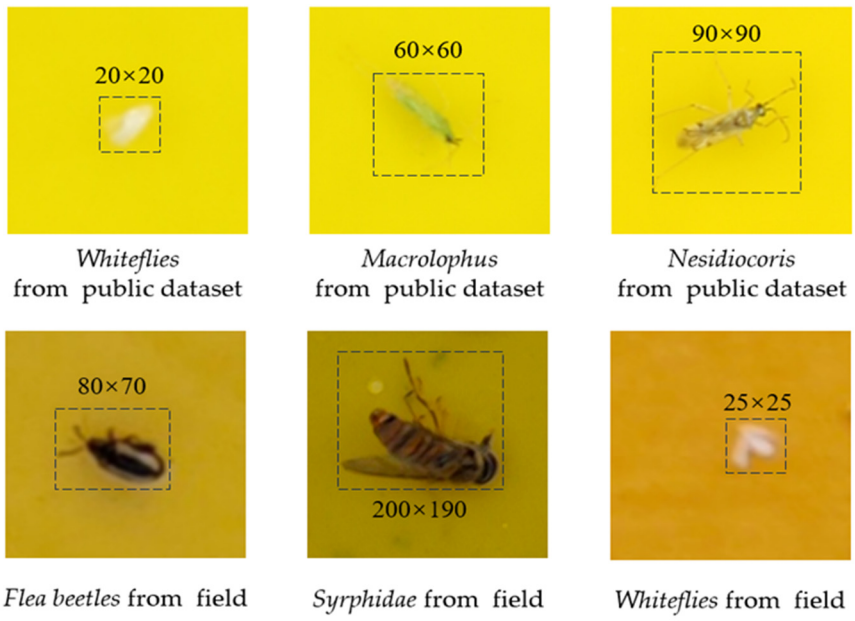

Figure 3. Pest on yellow sticky traps from field and public dataset.

2.3. Data Preprocessing

In this study, a total of 436 raw pest images were collected by placing yellow sticky traps in the field. After filtering the images collected from the yellow sticky traps, those with impurities, damage, or unclear conditions were removed, resulting in a total of 327 pest images. The images were annotated using the LabelImg tool, and the labels were saved in TXT format. The pest annotations were supported by experts Fanghao Wan and Bo Liu, who have extensive experience in the field of entomology, and were subsequently labeled by the researchers of this study, with a final review by the experts to ensure the accuracy of pest classification. During the labeling process, each pest was marked with the smallest possible bounding box to improve localization accuracy. To enhance the model's generalization and robustness, improve its ability to learn data features, and avoid overfitting, data augmentation was applied to the 327 collected images and the 284 public dataset images. The augmentation methods included vertical flipping, scaling, rotation, cropping, brightness adjustment, saturation adjustment, and noise addition, resulting in a total of 3527 images of pests on yellow sticky traps. Table 2 shows the dataset size for each pest category before and after data augmentation. Where the number is the number of pests of each species.

Table 2. The dataset size for each pest category.

Category	Before Data Augmentation	After Data Augmentation
Whiteflies (WF)	12,780	137,768
Macrolophus (MR)	8680	93,570
Nesidiocoris (NC)	8100	87,318
Flea beetles (SFB)	11,445	123,377
Syrphidae (SP)	6905	74,435

Figure 4 shows the effects of the data augmentation on the images. The dataset was divided into training, validation, and test sets in a 7:2:1 ratio, consisting of 2469 images for training, 706 for validation, and 352 for testing. To enable the model to better learn the pest features during training, both the training and validation sets included images from the

collected dataset and the public dataset, while the test set was kept separate to evaluate the model's generalization ability and performance on different datasets.

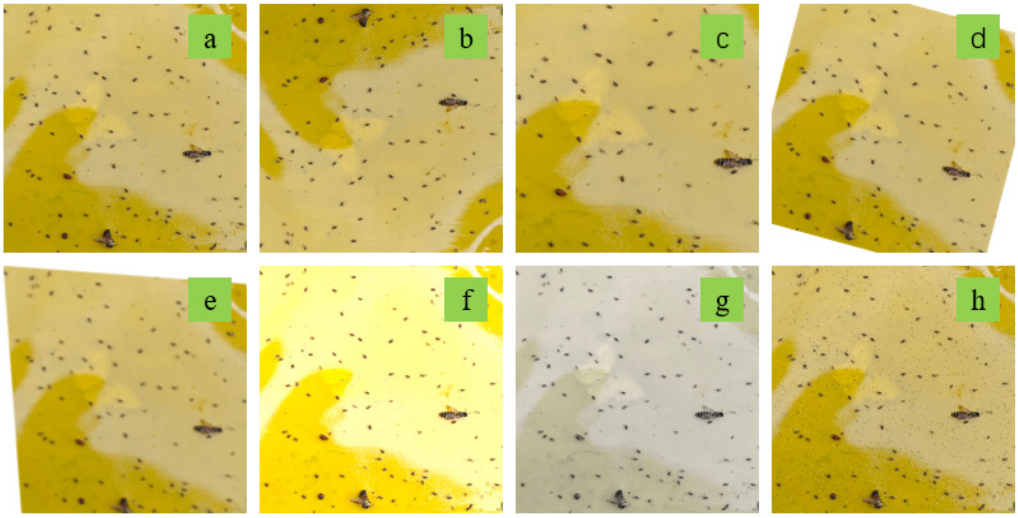

Figure 4. Data augmentation: (**a**) Original image; (**b**) flipping; (**c**) crop; (**d**) rotation; (**e**) shear; (**f**) increased brightness; (**g**) reduce saturation; (**h**) added noise.

2.4. Object Detection Model

2.4.1. YOLO Model

YOLO models have wide applications in the field of object detection. In 2016, researchers, including Joseph Redmon, Santosh Divvala, and Ross Girshick [43], introduced a one-stage object detection network, YOLOv1, which attracted widespread attention due to its extremely fast detection speed and unique design concept. To date, the YOLO series has evolved to the YOLOv11 version. Although the latest YOLO version, YOLOv11, shows significant improvements, it still performs poorly in detecting low-resolution images and small objects. YOLOv10, released in May 2024, advances the performance and efficiency of YOLO through improvements in both post-processing and model architecture. YOLOv10 achieves optimization over previous versions primarily through a consistency-based dual assignment strategy without NMS training and a comprehensive efficiency–accuracy driven model design. The NMS-free training strategy accelerates model inference speed, making deployment on edge computing platforms more efficient. The efficiency–accuracy model design utilizes intrinsic rank analysis to identify and reduce redundancies in model stages, while adopting a simplified architecture with CIB to minimize computational overhead and decouple spatial downsampling and channel expansion to reduce both computational cost and parameter count. Compared with other state-of-the-art detectors, YOLOv10 achieves cutting-edge performance with reduced latency.

2.4.2. YOLO-YSTs Model

This study selects the YOLOv10n model as the baseline for improvement. Firstly, the SPD-Conv module is used to replace the convolutional strides and pooling operations in the YOLOv10n model. SPD-Conv consists of a space-to-depth (SPD) layer and a convolution layer (non-strided convolution) and can be applied to most CNN architectures. SPD-Conv downsamples feature maps while preserving learnable information, completely eliminating traditional convolutional strides and pooling operations. SPD-Conv achieves its goal through several key steps: 1. Replacing convolutional strides and pooling layers: In traditional CNN architectures, convolutional strides and pooling layers reduce the spatial dimensions of the feature map to decrease computational load and expand the receptive

field. However, these operations often lead to a loss of detail information, which negatively impacts detection accuracy for small objects. SPD-Conv eliminates these operations to preserve more spatial information and avoid information loss. 2. Space-to-depth layer (SPD): This layer rearranges the elements of the feature map, transforming spatial information into the depth dimension, increasing the depth of the feature map while preserving all information in the channel dimension, thereby avoiding the information loss typically encountered in traditional downsampling methods. 3. Non-strided convolution layer: After the SPD layer, SPD-Conv introduces a convolution layer with a stride of 1 for feature extraction. This means that the convolution kernel slides over each pixel of the input feature map without skipping any positions. In this way, the convolution kernel processes every region of the feature map, ensuring maximum preservation of image information and generating richer feature representations. This reduces the number of channels in the feature map without losing details, while effectively processing the features. Through this mechanism, SPD-Conv retains more information during the feature extraction phase, thereby enhancing the model's ability to recognize small target images, making it suitable for detecting small pests on yellow sticky traps [34]. As shown in Figure 5, SPD-Conv is used to replace the original strided convolution layers in the YOLOv10n model to optimize feature extraction.

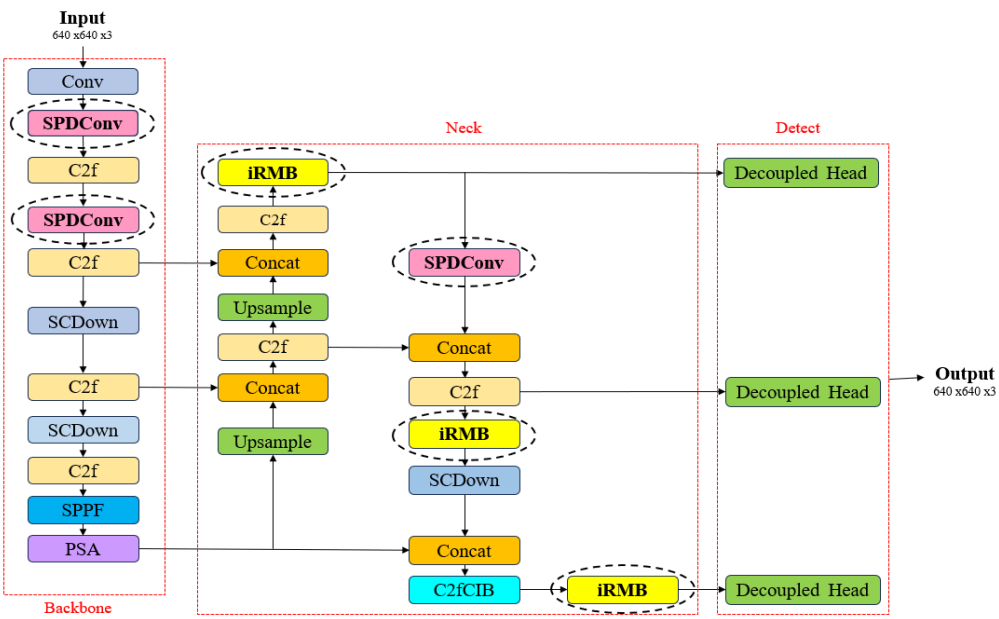

Figure 5. Structure of YOLO-YSTs.

Secondly, the iRMB inverted residual block attention mechanism is added to the YOLOv10n model to balance the model's focus on local detail features and global semantic features. The inverted residual block (IRB) is a basic module used in lightweight CNN structures, primarily designed to extract richer features by first expanding the number of channels and then performing feature extraction. Compared with traditional residual blocks, which compress the number of channels first, IRBs demonstrate better performance in some lightweight networks. However, no corresponding structure has been proposed for attention-based models. The iRMB extends the traditional CNN IRB design concept to an attention-based structure and combines the efficiency of CNN architectures in capturing local features with the long-range interaction modeling capability of transformer architectures, considering the entire input space during feature extraction, which effectively aids in global object detection. iRMB enables the model to maintain its lightweight nature while effectively utilizing computational resources and achieving high accuracy, thus enhancing

the model's ability to process long-distance information. Additionally, the meta-mobile block is abstracted to utilize various expansion ratios and efficient operators, achieving a modular design. Through a plug-and-play approach, different operations are integrated into a unified framework, providing the model with greater flexibility [37]. As shown in Figure 5, the iRMB attention mechanism is added to the YOLOv10n model to enhance the model's focus on pests in dense regions, while also making it suitable for deployment on edge mobile devices for pest detection.

Finally, to address the issue of missed detections and false positives for small and overlapping pests on yellow sticky traps, the Inner-SIoU loss function is used to replace the original CIoU loss function in the YOLOv10n model. Traditional IoU calculation methods consider the overall overlap between predicted and ground-truth bounding boxes. In contrast, Inner-IoU focuses on the core internal overlap of the bounding boxes rather than the entire overlap, providing a more precise evaluation of the overlapping regions. By introducing auxiliary bounding boxes to calculate the loss function, it can be integrated into existing IoU-based loss functions, offering strong generalization capabilities for various detection tasks [39]. The SIoU loss function integrates angle considerations, accounting for the angle between the expected and predicted regression directions, thereby redefining the penalty metric and overcoming the limitations of previous loss functions in region evaluation. The SIoU loss function consists of four components: angle loss, distance loss, shape loss, and IoU loss. This improvement effectively reduces the degrees of freedom, allowing the SIoU loss function to be easily incorporated into any object detection task, thereby facilitating better results [40]. This paper improves the YOLOv10n model by combining Inner-IoU and SIoU into the Inner-SIoU loss function, specifically targeting the challenge of detecting overlapping pests. The final network structure of the improved YOLO-YST is shown in Figure 5. The black dotted box indicates the position where the module is to be added in the network structure.

2.5. Experimental Setup

The experiments in this study were conducted on a Windows 11 (64-bit) system with an NVIDIA RTX 4070 GPU. The software environment includes Python 3.11, CUDA 12.4, and PyTorch 2.4.1. The experimental environment and specific parameter configurations are shown in Table 3.

Table 3. Experimental test platform.

Environment	Model Number	Parameters
Computer system	Windows 11	RAM: 64 GB
CPU	Intel i9-13900	Frequency: 2.00 GHz
GPU	NVIDIA RTX 4070	Video memory: 12 GB
Deep learning framework	PyTorch	2.4.1
Computational platform	CUDA	12.4
Software environment	Python	3.11

The input image size for model training was set to 640 × 640. By experimenting with various hyperparameter combinations, the batch size was set to 32, the initial learning rate to 0.01, and the number of training epochs to 200, with early stopping set at 100. GPU acceleration was used for training, as shown in Table 4.

Table 4. Training parameters.

Parameters	Value
Img-size	640 × 640
Batch-size	32
Initial learning rate	0.01
Epoch	200
Patience	100

2.6. Evaluation Index

This study evaluates the model's training results using precision (P), recall (R), mean average precision (mAP), and frames per second (FPS). Precision refers to the proportion of actual positives among the samples predicted as positive. Recall refers to the proportion of actual positives correctly predicted. mAP is the mean of the average precision (AP) for all classes, reflecting the overall accuracy of the model. FPS is used to evaluate the model's processing speed on the given hardware, which is crucial for real-time monitoring. The formulas for calculating these evaluation metrics are shown in Equations (1)–(4).

$$P = \frac{TP}{TP + FP} \tag{1}$$

$$R = \frac{TP}{TP + FN} \tag{2}$$

$$AP = \int_0^1 PR \cdot dr \tag{3}$$

$$mAP = \frac{1}{n} \sum_k^{k=n} AP_k \tag{4}$$

Here, *TP* refers to true positives, indicating the number of correctly predicted pest categories by the model. *FP* refers to false positives, indicating the number of instances where the model incorrectly predicts a non-pest category. *FN* refers to false negatives, indicating the number of instances where the model incorrectly predicts a pest category. n represents the number of classes and AP_k denotes the average precision for the *k*-th class.

3. Result

3.1. YOLO-YSTs Training Result

After 200 training epochs, a comprehensive evaluation of the YOLO-YSTs model's performance was conducted. Figure 6 illustrates the training process of the YOLO-YSTs model and the YOLOv10n model. As the number of iterations increased, the precision (P), recall (R), mAP_{50}, and $mAP_{50–95}$ metrics showed a clear trend of improvement. During the initial training period, P, R, $mAP_{50,}$ and $mAP_{50–95}$ were relatively low, reflecting the model's insufficient ability to learn the target features accurately. As the number of training epochs increased, these metrics gradually improved, indicating enhanced model performance. In the later stages of training, P, R, mAP_{50}, and $mAP_{50–95}$ stabilized, indicating that the model had accurately learned the target features and reached optimal performance. After 200 epochs, mAP_{50} reached 0.868, $mAP_{50–95}$ reached 0.423, and both P and R achieved 0.832. Compared with the YOLOv10n model, the improved model demonstrates better overall performance and faster convergence. These results indicate that our model successfully fitted the pest detection task.

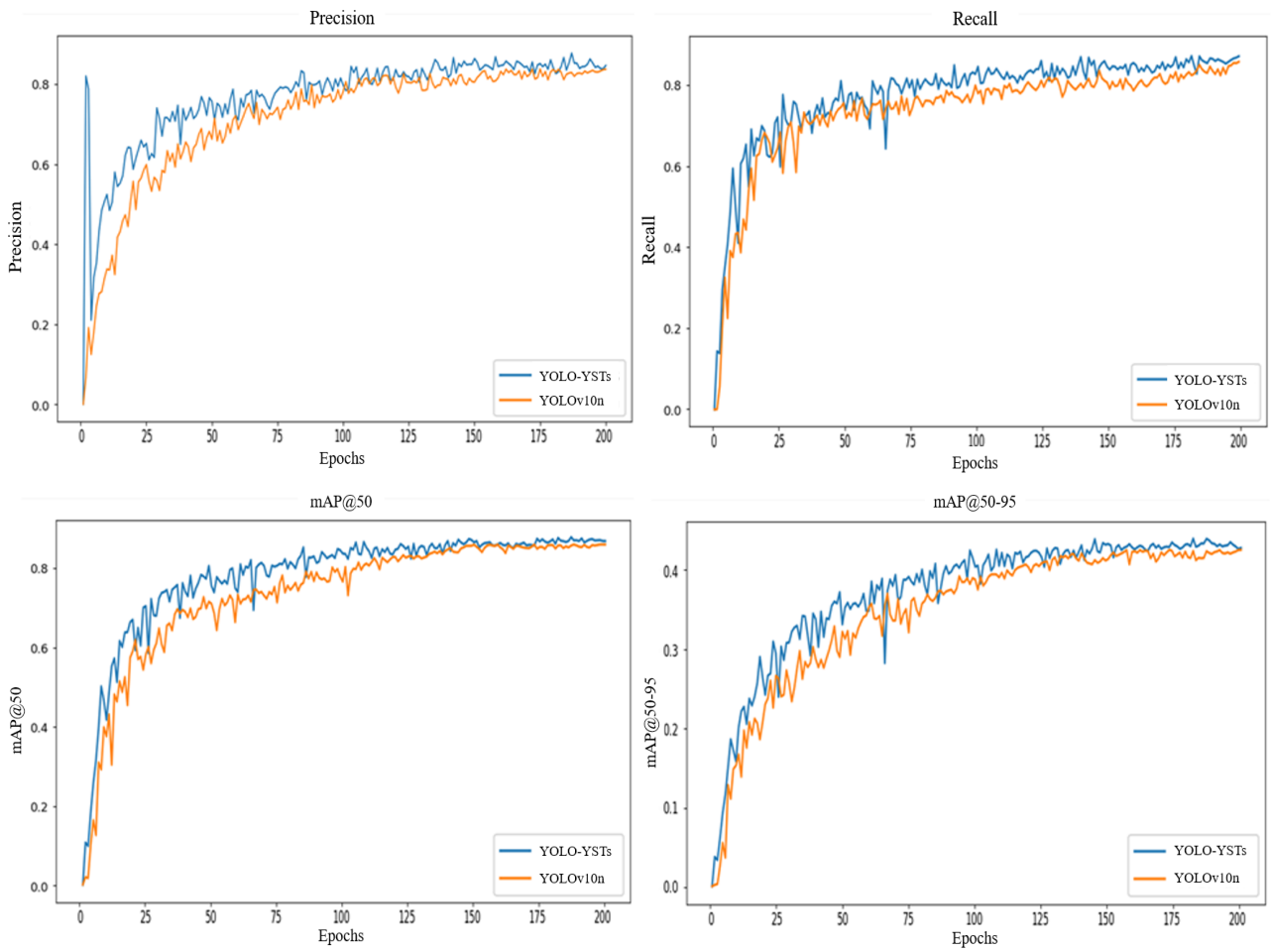

Figure 6. Training process curves of YOLO-YSTs and YOLOv10n.

Figure 7 shows some of the detection results from the common dataset. Both the YOLOv10n and YOLO-YSTs models are able to detect pest images on yellow sticky traps. However, YOLOv10n exhibits issues of missed and false detections for certain targets. In (a), for the overlapping *Whiteflies* (WF) on the left side of the image, the YOLOv10n model incorrectly identified the target as *Macrolophus* (MR), while the YOLO-YSTs model correctly identified the overlapping target. In (b), for the detection of *Nesidiocoris* (NC), the YOLOv10n model missed the target in the lower-right corner, while the proposed model successfully detected it.

Figure 8 shows some of the detection results from our dataset and (b) represents the highest number of pests on yellow sticky traps in the wild. In (a), for the detection of *Syrphidae* (SP), the YOLOv10n model failed to detect the target, while the YOLO-YSTs model successfully identified it. In (b), for the densely distributed *Flea beetles* (SFB), the YOLOv10n model exhibited significant missed detections, while the YOLO-YSTs model resolved the issue of detecting dense targets. In (c), the YOLOv10n model misidentified other pest species as *Syrphidae* (SP), while YOLO-YSTs avoided false detections. In summary, the YOLO-YSTs model enhances the detection capability for small and dense targets, successfully identifying small targets on yellow sticky traps and effectively addressing the issues of missed and false detections.

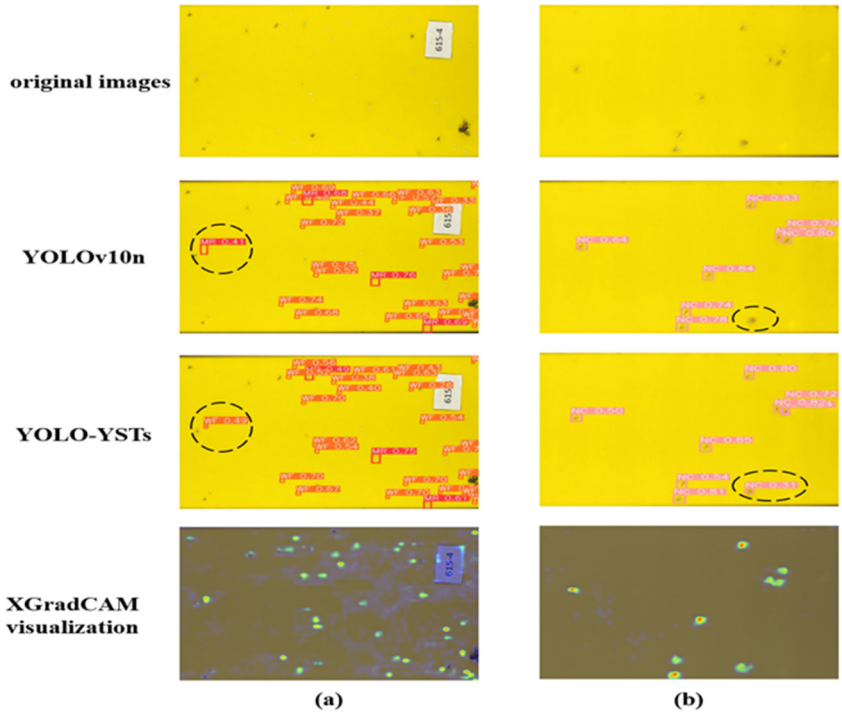

Figure 7. Comparison of detection effect in the public dataset: (**a**) False detection of overlapping pests; (**b**) missing detection of *Nesidiocoris*.

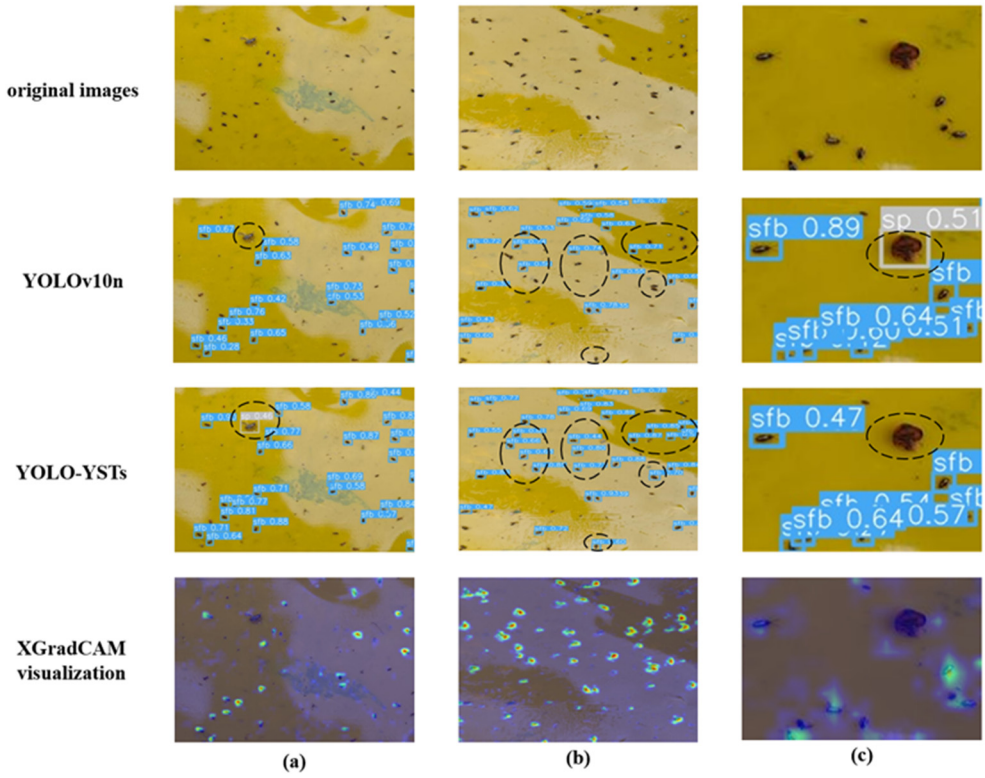

Figure 8. Comparison of detection effect in field dataset: (**a**) Missing detection of *Syrphidae*; (**b**) missing detection of *Flea beetles*; (**c**) false detection of *Syrphidae*.

3.2. Comparison with Other Mainstream Detection Models

3.2.1. Comparisons with Other YOLO Versions

To validate the advantages of the YOLO-YSTs model, it was comprehensively compared with other mainstream lightweight models, as shown in Table 5. It can be observed

that the YOLO-YSTs model achieved a precision of 83.2% and an mAP_{50} of 86.8% on the yellow sticky trap dataset, both of which outperform other object detection models. This indicates that the YOLO-YSTs model achieves optimal detection accuracy among YOLO series models. Compared with the original YOLOv10n model, the YOLO-YSTs model improved mAP_{50} by 1.7% while maintaining a high FPS and keeping the GFLOPs nearly the same. Although YOLOv5n, YOLOv8n, and YOLOv11n models have higher FPS and lower GFLOPs, their precision and mAP_{50} are inferior to those of the YOLO-YSTs model, indicating lower detection accuracy. The YOLO-YSTs model has 8.8 GFLOPs, only 0.1 higher than the original model, yet its mAP_{50} increased by 1.7%. Moreover, the weight file generated by YOLO-YSTs after training is only 6.5MB. These results demonstrate that YOLO-YSTs strikes a balance between model size and detection accuracy, with a significant advantage in inference speed, making it highly suitable for real-time detection on edge devices.

Table 5. Comparisons with other YOLO versions.

Model	P	R	mAP_{50}	$mAP_{50–95}$	GFLOPs	FPS
YOLOv5n	82.8%	81.9%	83.7%	39.9%	7.3	167
YOLOv6n	81.5%	72.4%	79.6%	38.5%	11.1	85
YOLOv7-tiny	76.8%	72.9%	78.4%	36.0%	13.2	81
YOLOv8n	80.2%	82.9%	84.5%	38.9%	8.1	156
YOLOv9-t	80.8%	83.7%	83.9%	38.2%	10.7	93
YOLOv10n	82.3%	81.9%	85.1%	40.2%	8.7	142
YOLOv11n	82.1%	83.5%	84.0%	38.3%	6.3	173
YOLO-YSTs (Ours)	83.2%	83.2%	86.8%	41.3%	8.8	139

3.2.2. Comparisons with Other SOTA Models

To validate the model's advantages, this paper compared the YOLO-YSTs model with other state-of-the-art (SOTA) models, such as faster R-CNN and SSD, on the detection of five pest categories, using mAP_{50} as the evaluation metric, as shown in Table 6. Although the SSD algorithm shows similar recognition accuracy across categories, its mAP_{50} is very low, indicating poor overall detection performance for the five pest categories. Faster R-CNN achieved a detection accuracy of above 80% for *Whiteflies* (WF), *Nesidiocoris* (NC), and *Flea beetles* (SFB), its detection performance for *Syrphidae* (SP) was suboptimal. The proposed YOLO-YSTs model outperforms other models in detection accuracy for each pest category. Therefore, the YOLO-YSTs model exhibits superior detection capabilities compared with other models, achieving high detection accuracy across all categories.

Table 6. Object class-wise mAP_{50} across different SOTA models.

Model	WF	MR	NC	SFB	SP
Faster R-CNN	81.5%	79.6%	83.7%	81.8%	68.5%
SSD	57.4%	54.8%	63.5%	61.9%	52.2%
YOLO-YSTs(Ours)	89.4%	86.7%	91.3%	95.3%	75.7%

3.3. Ablation Experiment

To verify the optimization effects of the SPD-Conv, iRMB, and Inner-SIoU modules, YOLOv10n was used as the baseline model, and ablation experiments were conducted under the same experimental conditions, as shown in Table 7. By replacing the original model's stride convolutions and pooling layers with the SPD-Conv module, the number of parameters and computational complexity of the convolutional layers were reduced, leading to a decrease in GFLOPs and parameter count, while avoiding information loss,

which resulted in a 0.7% improvement in mAP_{50}. After adding the iRMB inverted residual block attention mechanism, although GFLOPs and parameter count increased, the model's attention to local features was improved, resulting in a 0.2% increase in mAP_{50}. After using the Inner-SIoU loss function, a more accurate evaluation of the overlapping regions was achieved, leading to a 0.5% increase in mAP_{50} without adding computational complexity. It can be observed that each of the three modules contributes to an increase in mAP_{50}, with SPD-Conv performing the best. Among the combinations of any two modules, the combination of SPD-Conv and iRMB performed the best. When all three modules were integrated into the model's network architecture, GFLOPs only increased by 0.1, yet detection accuracy improved by 1.7%, reaching the highest level and demonstrating the best overall performance compared with other models, while ensuring the model's lightweight nature. The results indicate that the SPD-Conv module, iRMB inverted residual block attention mechanism, and Inner-SIoU loss function are compatible and collectively enhance the model's performance. In resource-constrained environments, these improvements significantly boost the model's detection accuracy for pests on yellow sticky traps, while maintaining its lightweight nature.

Table 7. Ablation test and analysis of results.

Model	P	R	mAP_{50}	$mAP_{50–95}$	GFLOPs	Params (M)
YOLOv10n	82.3%	81.9%	85.1%	40.2%	8.7	2.77
SPD-Conv	81.4%	82.6%	85.8%	41.1%	8.4	2.74
iRMB	81.1%	81.4%	85.3%	40.8%	9.4	3.12
Inner-SIoU	80.9%	82.9%	85.6%	40.6%	8.7	2.77
SPD + IRMB	82.4%	81.0%	85.9%	40.1%	9.2	2.85
SPD + Inner-SIou	80.2%	81.1%	85.3%	39.9%	8.5	2.74
iRMB + Inner-SIou	82.1%	82.7%	85.5%	41.0%	8.9	2.93
YOLO-YSTs	83.2%	83.2%	86.8%	41.3%	8.8	3.02

3.4. Model Deployment and Real-Time Detection

To further validate the effectiveness of the proposed lightweight model on edge devices, the YOLO-YSTs model was deployed and tested on the Raspberry Pi 4B. To accelerate inference on the Raspberry Pi 4B, NCNN is used for inference. NCNN is a high-performance neural network forward computation framework developed by Tencent Youtu Lab, primarily designed for mobile and embedded devices. It supports multiple platforms, including Linux, Android, iOS, and Windows, and can run on various hardware architectures. NCNN also supports models exported from multiple deep learning frameworks, including TensorFlow, PyTorch, and ONNX, and is capable of processing common convolutional neural network models to achieve efficient inference. As shown in Figure 9, the deployment process involves first converting the trained PyTorch model to the intermediate ONNX format, then compiling NCNN on the Raspberry Pi 4B. Next, the ONNX file is transferred to the Raspberry Pi 4B, and the onnx2ncnn tool is used to convert it into NCNN format. Finally, the model performs inference on data captured by the camera. For example, Jani et al. [44] deployed various models on the Raspberry Pi for real-time inference and tested them on the pest dataset, achieving good detection results.

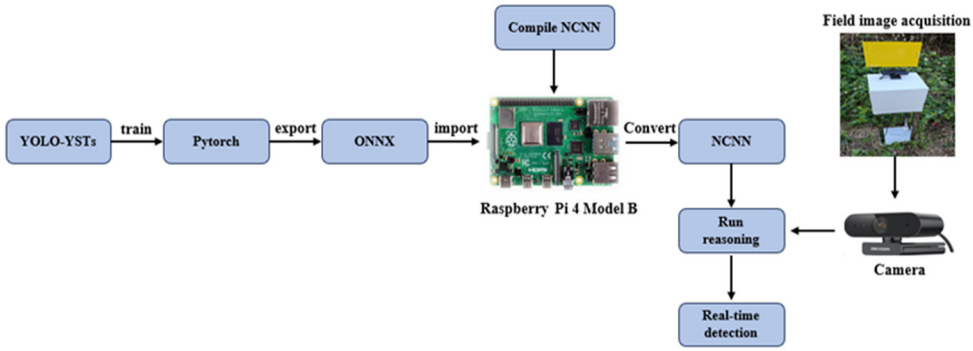

Figure 9. Edge deployment process.

Figure 10 shows the detection speed of the YOLO-YSTs model deployed on the Raspberry Pi 4B in PyTorch, ONNX, and NCNN formats. As shown in the figure, the model's detection speed improved to 22 FPS when using the NCNN format.

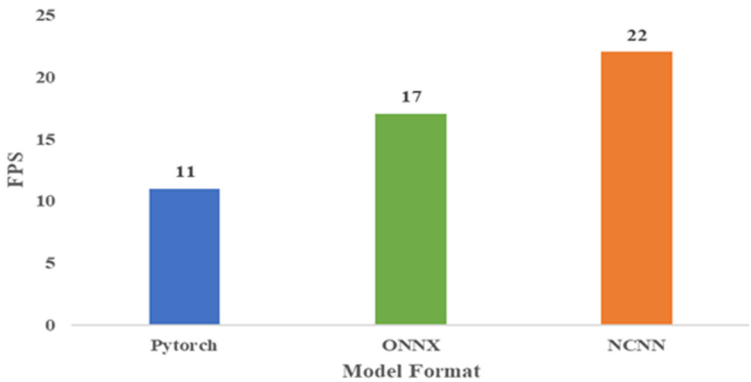

Figure 10. Detection speed on the Raspberry Pi 4B.

Figure 11 shows the field simulation of the pest detection device and the real-time detection results of pest images obtained using the model deployed on the Raspberry Pi 4B. The results indicate that, even with small and densely distributed pest targets on the yellow sticky trap, the model still achieved good detection performance, successfully detecting pests on the trap.

(a) (b)

Figure 11. Field detection effect: (**a**) Pest detection device; (**b**) real-time detection screen on Raspberry Pi 4B.

In conclusion, the YOLO-YSTs model proposed in this paper meets the requirements for pest detection in complex agricultural field environments on edge devices, providing technical support for on-site pest intelligent monitoring.

4. Discussion

Deep learning technology has shown remarkable effectiveness in the field of pest detection and is widely applied, making it one of the key tools for pest detection. In related studies applying YOLO object detection models, some have achieved high-precision detection of small and densely packed targets. For example, Liu et al. [45] improved the YOLOv10 model to successfully enhance the detection accuracy of densely distributed tomatoes, demonstrating the feasibility of improvements to the YOLOv10 model. Xue et al. [46] enhanced the YOLOv5 model to improve detection accuracy for small-target tree pests while achieving model lightweighting, demonstrating the feasibility of YOLO series models in pest detection and high-performance lightweight models. These studies demonstrate that the results of this research are not coincidental. This paper aims to utilize deep learning-based object detection algorithms to establish a pest dataset composed of yellow sticky trap pest images captured in a device acquisition environment simulated by smartphones, alongside images from public datasets. The dataset was used to train current mainstream lightweight object detection models, demonstrating the recognition performance of pests in the images. Due to the complexity of field environments, deploying high-performance large-scale equipment in the field is challenging. Therefore, edge mobile platforms demonstrate their convenient advantages in such environments. Scholars have also conducted research on using edge mobile platforms for pest detection and have validated their feasibility [47]. However, the limited computational resources of edge mobile platforms make lightweight object detection models a necessary requirement for deployment on such devices, while ensuring that detection accuracy is not compromised.

To address these challenges, this paper proposes the YOLO-YSTs model for pest detection on yellow sticky traps. The model is characterized by high detection accuracy and fast processing speed. YOLO-YSTs achieves a detection accuracy of 86.8% in $mAP_{50,}$ with a GFLOPs of only 8.8. The YOLO-YSTs model was then compared with current mainstream lightweight object detection algorithms in experimental tests. In comparison, the YOLO-YSTs model outperforms other models in detection accuracy, and its FPS meets real-time detection requirements. Finally, the YOLO-YSTs model was deployed on the Raspberry Pi 4B, achieving accurate and fast pest detection on yellow sticky traps.

Although this study has achieved certain preliminary results, some limitations still exist. Firstly, the number of yellow sticky trap pest images and pest species collected in this study is still insufficient, and large datasets are critical for improving model accuracy, thus limiting the model's performance. In future work, the variety of pests will be expanded, and more pest images will be collected from the field to improve the performance of the proposed model. Secondly, the pest detection device developed is relatively simple. Future work will focus on optimizing the device, using rapid prototyping technologies such as 3D printing to efficiently manufacture the components, enabling quick assembly into a final product. Finally, although the proposed model addresses the issues of missed and false detections in the original model, its performance on detecting severely deformed or fragmented pests is still inadequate, resulting in continued missed and false detections. Future work will focus on further optimizing the model to resolve this issue.

5. Conclusions

This paper proposes a lightweight object detection model, YOLO-YSTs, to address the challenge of accurately and automatically identifying pests on yellow sticky traps

in complex agricultural environments, providing technical support for intelligent pest monitoring. Building upon the YOLOv10n model, this paper utilizes the SPD-Conv module to replace the original stride convolution and pooling operations, allowing the model to focus on the challenging task of detecting dense objects while reducing its parameters. Additionally, the iRMB inverted residual block attention mechanism is incorporated into the YOLOv10n model, balancing model parameters, efficiency, and accuracy. Finally, the use of the Inner-SIoU loss function in the YOLOv10n model enhances its ability to detect small objects. To validate the model's effectiveness on edge computing devices, the YOLO-YSTs model was deployed on a Raspberry Pi 4B, with accelerated inference via NCNN, enabling real-time pest detection on yellow sticky traps. The proposed YOLO-YSTs model achieves an mAP_{50} of 86.8% and a precision of 83.2%, demonstrating high recognition accuracy for five pest species. The optimal weights obtained after training are only 6.5MB, with the model parameters and GFLOPs being 3.02M and 8.8, respectively. The GFLOPs increased by only 0.1 compared with the YOLOv10n model, yet the detection accuracy improved by 1.7%, achieving higher precision. The proposed model provides significant technical assistance for the development of automation and intelligence in field pest monitoring. In future work, edge computing platforms can be deployed in the field, coupled with mechanical devices, to achieve long-term automatic pest identification and monitoring, providing a data foundation for high-quality pest control and the promotion of sustainable agricultural development.

Author Contributions: Y.H.: writing—original draft, methodology, formal analysis; Z.L. (Zhenhao Liu): writing—review and editing, validation, software, methodology, formal analysis, data curation; H.Z.: methodology, resources; C.T.: funding acquisition, resources; B.L.: writing—review and editing, resources; Z.L. (Zaiyuan Li): writing—review and editing, supervision; F.W.: resources, investigation; W.Q.: funding acquisition, supervision, resources; X.Q.: funding acquisition, project administration, resources, writing—review and editing. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by Shenzhen Science and Technology Program (KCXFZ-20240903093859009), Science and Technology special fund of Hainan Province (ZDYF2022XDNY214), and the Agricultural Science and Technology Innovation Program.

Data Availability Statement: The raw/processed data required to reproduce the above findings cannot be shared at this time, as the data also form part of an ongoing study.

Acknowledgments: Acknowledgments for the data support from Chinese Academy of Agricultural Sciences.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Dieckhoff, C.; Meyhöfer, R. If Only You Could Catch Me—Catch Me If You Can: Monitoring Aphids in Protected Cucumber Cultivations by Means of Sticky Traps. *Horticulturae* **2023**, *9*, 571. [CrossRef]
2. Carrillo-Arámbula, L.; Infante, F.; Cavalleri, A.; Gómez, J.; Ortíz, J.; Fanson, B.; González, F. Colored sticky traps for monitoring phytophagous thrips (Thysanoptera) in mango agroecosystems, and their impact on beneficial insects. *PLoS ONE* **2022**, *17*, e0276865. [CrossRef] [PubMed]
3. Heinz, K.; Parrella, M.; Newman, J. Time-Efficient Use of Yellow Sticky Traps in Monitoring Insect Populations. *J. Econ. Èntomol.* **1992**, *85*, 2263–2269. [CrossRef]
4. Mohammadipour, A. The role of sticky yellow traps in reducing the population of the Dubas Bug, *Ommatissus lybicus*. *J. Entomol. Soc. Iran* **2024**, *1403*, 407–415. [CrossRef]
5. Pathan, S.; Solangi, A.; Marri, J.; Khuhro, S.; Lanjar, A. Management of *Scriptothrip dorsalis* (Thysanoptera; Thripidae) in Chilli Crop. *J. Appl. Res. Plant Sci.* **2023**, *4*, 637–641. [CrossRef]
6. Pinto-Zevallos, D.; Vänninen, I. Yellow sticky traps for decision-making in whitefly management: What has been achieved? *Crop Prot.* **2013**, *47*, 74–84. [CrossRef]

7. Moreau, T.; Isman, M. Combining reduced-risk products, trap crops and yellow sticky traps for greenhouse whitefly (*Trialeurodes vaporariorum*) management on sweet peppers (*Capsicum annum*). *Crop Prot.* **2012**, *34*, 42–46. [CrossRef]
8. Böckmann, E.; Meyhöfer, R. Sticky trap monitoring of a pest-predator system in glasshouse tomato crops: Are available trap colours sufficient? *J. Appl. Entomol.* **2017**, *141*, 339–351. [CrossRef]
9. Böckmann, E.; Pfaff, A.; Schirrmann, M.; Pflanz, M. Rapid and low-cost insect detection for analysing species trapped on yellow sticky traps. *Sci. Rep.* **2021**, *11*, 12854.
10. Xia, D.; Chen, P.; Wang, B.; Zhang, J.; Xie, C. Insect Detection and Classification Based on an Improved Convolutional Neural Network. *Sensors* **2018**, *18*, 4169. [CrossRef]
11. Gutierrez, A.; Ansuategi, A.; Susperregi, L.; Tubío, C.; Rankic, I.; Lenža, L. A Benchmarking of Learning Strategies for Pest Detection and Identification on Tomato Plants for Autonomous Scouting Robots Using Internal Databases. *Sensors* **2019**, *2019*, 5219471. [CrossRef]
12. Ramalingam, B.; Mohan, R.; Pookkuttath, S.; Gómez, B.; Borusu, C.; Teng, T.; Tamilselvam, Y. Remote Insects Trap Monitoring System Using Deep Learning Framework and IoT. *Sensors* **2020**, *20*, 5280. [CrossRef] [PubMed]
13. Deserno, M.; Briassouli, A. Faster R-CNN and EfficientNet for Accurate Insect Identification in a Relabeled Yellow Sticky Traps Dataset. In Proceedings of the 2021 IEEE International Workshop on Metrology for Agriculture and Forestry (MetroAgriFor), Trento-Bolzano, Italy, 3 November 2021; pp. 209–214.
14. Xia, C.L.; Chon, T.S.; Ren, Z.M.; Lee, J.M. Automatic identification and counting of small size pests in greenhouse conditions with low computational cost. *Ecol. Inform.* **2015**, *29*, 139–146. [CrossRef]
15. Domingues, T.; Brandão, T.; Ribeiro, R.; Ferreira, J.C. Insect Detection in Sticky Trap Images of Tomato Crops Using Machine Learning. *Agriculture* **2022**, *12*, 1967. [CrossRef]
16. Yang, S.; Zhou, G.; Feng, Y.; Zhang, J.; Jia, Z. SRNet-YOLO: A model for detecting tiny and very tiny pests in cotton fields based on super-resolution reconstruction. *Front. Plant Sci.* **2024**, *15*, 1416940. [CrossRef]
17. Wen, C.; Chen, H.; Ma, Z.; Zhang, T.; Yang, C.; Su, H.; Chen, H. Pest-YOLO: A model for large-scale multi-class dense and tiny pest detection and counting. *Front. Plant Sci.* **2022**, *13*, 973985. [CrossRef]
18. Zhang, W.; Huang, H.; Sun, Y.; Wu, X. AgriPest-YOLO: A rapid light-trap agricultural pest detection method based on deep learning. *Front. Plant Sci.* **2023**, *13*, 1079384. [CrossRef]
19. Yang, S.; Xing, Z.; Wang, H.; Dong, X.; Gao, X.; Liu, Z.; Zhang, X.; Li, S.; Zhao, Y. Maize-YOLO: A New High-Precision and Real-Time Method for Maize Pest Detection. *Insects* **2023**, *14*, 278. [CrossRef]
20. Liu, J.; Wang, X. Tomato Diseases and Pests Detection Based on Improved Yolo V3 Convolutional Neural Network. *Front. Plant Sci.* **2020**, *11*, 898. [CrossRef]
21. Ahmad, I.; Yang, Y.; Yue, Y.; Ye, C.; Hassan, M.; Cheng, X.; Wu, Y.; Zhang, Y. Deep Learning Based Detector YOLOv5 for Identifying Insect Pests. *Appl. Sci.* **2022**, *12*, 10167. [CrossRef]
22. Huang, J.; Huang, Y.; Huang, H.; Zhu, W.; Zhang, J.; Zhou, X. An Improved YOLOX Algorithm for Forest Insect Pest Detection. *Comput. Intell. Neurosci.* **2022**, *2022*, 5787554. [CrossRef] [PubMed]
23. Di, Y.; Phung, S.; Van Den Berg, J.; Clissold, J.; Bouzerdoum, A. TP-YOLO: A Lightweight Attention-Based Architecture for Tiny Pest Detection. In Proceedings of the 2023 IEEE International Conference on Image Processing (ICIP), Kuala Lumpur, Malaysia, 8 October 2023; pp. 3394–3398.
24. Jia, L.; Wang, T.; Chen, Y.; Zang, Y.; Li, X.; Shi, H.; Gao, L. MobileNet-CA-YOLO: An Improved YOLOv7 Based on the MobileNetV3 and Attention Mechanism for Rice Pests and Diseases Detection. *Agriculture* **2023**, *13*, 1285. [CrossRef]
25. Chen, J.; Lin, W.; Cheng, H.; Hung, C.; Lin, C.; Chen, S. A Smartphone-Based Application for Scale Pest Detection Using Multiple-Object Detection Methods. *Electronics* **2021**, *10*, 372. [CrossRef]
26. Zhang, L.; Zhao, C.; Feng, Y.; Li, D. Pests Identification of IP102 by YOLOv5 Embedded with the Novel Lightweight Module. *Agronomy* **2023**, *13*, 1583. [CrossRef]
27. Khalid, S.; Oqaibi, H.; Aqib, M.; Hafeez, Y. Small Pests Detection in Field Crops Using Deep Learning Object Detection. *Sustainability* **2023**, *15*, 6815. [CrossRef]
28. Xu, L.; Shi, X.; Tang, Z.; He, Y.; Yang, N.; Zheng, C.; Chen, H.; Zhou, T.; Huang, P.; Wu, Z.; et al. ASFL-YOLOX: An adaptive spatial feature fusion and lightweight detection method for insect pests of the *Papilionidae* family. *Front. Plant Sci.* **2023**, *14*, 1176300. [CrossRef]
29. Hu, Y.; Deng, X.; Lan, Y.; Chen, X.; Long, Y.; Liu, C. Detection of Rice Pests Based on Self-Attention Mechanism and Multi-Scale Feature Fusion. *Insects* **2023**, *14*, 280. [CrossRef]
30. Sun, H.; Nicholaus, I.; Fu, R.; Kang, D. YOLO-FMDI: A Lightweight YOLOv8 Focusing on a Multi-Scale Feature Diffusion Interaction Neck for Tomato Pest and Disease Detection. *Electronics* **2024**, *13*, 2974. [CrossRef]
31. Wang, A.; Chen, H.; Liu, L.; Chen, K.; Lin, Z.; Han, J.; Ding, G. YOLOv10: Real-Time End-to-End Object Detection. *arXiv* **2024**, arXiv:2405.14458.

32. Tan, L.; Liu, S.; Gao, J.; Liu, X.; Chu, L.; Jiang, H. Enhanced Self-Checkout System for Retail Based on Improved YOLOv10. *J. Imaging* **2024**, *10*, 248. [CrossRef]
33. Guan, S.; Lin, Y.; Lin, G.; Su, P.; Huang, S.; Meng, X.; Liu, P.; Yan, J. Real-Time Detection and Counting of Wheat Spikes Based on Improved YOLOv10. *Agronomy* **2024**, *14*, 1936. [CrossRef]
34. Sunkara, R.; Luo, T. *No More Strided Convolutions or Pooling: A New CNN Building Block for Low-Resolution Images and Small Objects*; Springer: Berlin/Heidelberg, Germany, 2023; Volume 13715, pp. 443–459.
35. Sun, R.; Fan, H.; Tang, Y.; He, Z.; Xu, Y.; Wu, E. Research on small target detection algorithm for UAV inspection scene based on SPD-conv. In Proceedings of the Fourth International Conference on Computer Vision and Data Mining (ICCVDM 2023), Changchun, China, 19 February 2023; pp. 686–691.
36. Peng, Y.; Yue, J. Road Crack Detection Algorithm Based on Improved YOLOv8. In Proceedings of the 2023 5th International Conference on Artificial Intelligence and Computer Applications (ICAICA), Dalian, China, 28–30 November 2023; pp. 28–32.
37. Zhang, J.; Li, X.; Li, J.; Liu, L.; Xue, Z.; Zhang, B.; Jiang, Z.; Huang, T.; Wang, Y.; Wang, C. Rethinking Mobile Block for Efficient Attention-based Models. In Proceedings of the 2023 IEEE/CVF International Conference on Computer Vision (ICCV), Paris, France, 1–6 October 2023; pp. 1389–1400.
38. Lu, S.; Xiao, Y.; Wang, G. Research on paint defect detection algorithm based on improved YOLOv5s. In Proceedings of the 2024 5th International Conference on Computer Engineering and Application (ICCEA), Hangzhou, China, 12–14 April 2024; pp. 1455–1459.
39. Zhang, H.; Xu, C.; Zhang, S. Inner-IoU: More Effective Intersection over Union Loss with Auxiliary Bounding Box. *arXiv* **2023**, arXiv:2311.02877.
40. Gevorgyan, Z. SIoU Loss: More Powerful Learning for Bounding Box Regression. *arXiv* **2022**, arXiv:2205.12740.
41. Chen, J.; Xie, Y.; Qian, Z.; Chen, K.; Zhen, M.; Hu, X. Power Adapter Appearance Defect Detection Based on Task Feature Decoupling YOLOv8n. *IEEE Access* **2024**, *12*, 60070–60080. [CrossRef]
42. Sun, G.; Liu, S.; Luo, H.; Feng, Z.; Yang, B.; Luo, J.; Tang, J.; Yao, Q.; Xu, J. Intelligent Monitoring System of Migratory Pests Based on Searchlight Trap and Machine Vision. *Front. Plant Sci.* **2022**, *13*, 897739. [CrossRef]
43. Redmon, J.; Divvala, S.; Girshick, R.; Farhadi, A. You Only Look Once: Unified, Real-Time Object Detection. In Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 27–30 June 2016; pp. 779–788.
44. Jani, D.; Patel, M.; Gajjar, R.; Gajjar, N. Comparative Analysis of Machine Learning Models for Pest Detection on Raspberry Pi. In Proceedings of the 2023 7th International Conference on Trends in Electronics and Informatics (ICOEI), Tirunelveli, India, 11–13 April 2023; pp. 991–995.
45. Liu, W.; Wang, S.; Gao, X.; Yang, H. A Tomato Recognition and Rapid Sorting System Based on Improved YOLOv10. *Machines* **2024**, *12*, 689. [CrossRef]
46. Xue, Z.; Xu, R.; Bai, D.; Lin, H. YOLO-Tea: A Tea Disease Detection Model Improved by YOLOv5. *Forests* **2023**, *14*, 415. [CrossRef]
47. Garcia, R.; Bicol, M.; Cababat, A.; Pontigon, J. A Raspberry Pi Microcontroller-based Insect Pests Detection, Counting and Logging System in Eggplants using SSD Lite MobileNetV2. In Proceedings of the 2021 IEEE 13th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment, and Management (HNICEM), Manila, Philippines, 28–30 November 2021; pp. 1–6.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.